



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 10
по курсу «Языки и методы программирования»
«Iterator. Бинарное отношение в C++»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Теоретические сведения	1
Класс BinaryRelation	1
relation.hpp	2
Итератор	4
Конструктор	4
advance_to_next()	4
Операторы	5
main()	6
Вывод	6

Цель работы

Реализовать шаблонный класс BinaryRelation с пользовательским итератором для обхода бинарного отношения. Использовать стандартные методы STL для итераторов.

Задачи:

- Реализовать шаблонный класс BinaryRelation.
- Создать пользовательский итератор.
- Реализовать begin() и end() для for-each.
- Проверить работу с бинарным отношением.

Теоретические сведения

Бинарное отношение — это подмножество декартова произведения $A \times B$. В данном случае отношение на множестве $0..N-1$, представленное булевой матрицей смежности.

Итератор — это объект, который позволяет перебирать элементы коллекции. Для использования в for-each нужно реализовать:

- begin() — возвращает итератор на первый элемент
- end() — возвращает итератор на элемент после последнего
- Операторы сравнения == и !=
- Оператор инкремента ++

Класс BinaryRelation

Шаблонный класс BinaryRelation с размером N.

relation.hpp

```

template <unsigned int N> class BinaryRelation {
    bool matrix[N][N];

public:
    BinaryRelation(const bool input[N][N]) {
        for (unsigned int i = 0; i < N; ++i)
            for (unsigned int j = 0; j < N; ++j)
                matrix[i][j] = input[i][j];
    }

    class iterator {
    private:
        const BinaryRelation *parent;
        std::pair<unsigned int, unsigned int> coords;
        unsigned int x, y;

    void advance_to_next() {
        while (x < N) {
            while (y < N) {
                if (parent->matrix[x][y]) {
                    coords = {x, y};
                    return;
                }
                ++y;
            }
            y = 0;
            ++x;
        }
        x = N;
        y = 0;
    }

    public:
        iterator(const BinaryRelation<N> *parent, unsigned int x,
            unsigned int y)
            : parent(parent), x(x), y(y), coords({x, y}) {
            advance_to_next();
        }

        std::pair<unsigned int, unsigned int> &operator deref()
        { return coords; }
        std::pair<unsigned int, unsigned int> *operator arrow()
        { return &coords; }

        bool operator eq(const iterator &other) const {
            return parent == other.parent && x == other.x && y ==
other.y;
        }
        bool operator ne(const iterator &other) const { return !(*this

```

- matrix — булева матрица N x N
- iterator — вложенный класс итератора

Итератор

Внутренний класс iterator обеспечивает обход бинарного отношения (где matrix[x][y] == true).

Конструктор

relation.hpp

```
iterator(const BinaryRelation<N> *parent, unsigned int x, unsigned
int y)
    : parent(parent), x(x), y(y), coords({x, y}) {
    advance_to_next();
}
```

Конструктор принимает начальные координаты и сразу переходит к первому элементу.

advance_to_next()

relation.hpp

```
void advance_to_next() {
    while (x < N) {
        while (y < N) {
            if (parent->matrix[x][y]) {
                coords = {x, y};
                return;
            }
            ++y;
        }
        y = 0;
        ++x;
    }
    x = N;
    y = 0;
}
```

Переходит к следующей паре, где matrix[x][y] == true.

Операторы

relation.hpp

```
std::pair<unsigned int, unsigned int> &operator deref() { return  
coords; }  
std::pair<unsigned int, unsigned int> *operator arrow() { return  
&coords; }  
  
bool operator eq(const iterator &other) const {  
    return parent == other.parent && x == other.x && y == other.y;  
}  
bool operator ne(const iterator &other) const { return !(*this ==  
other); }  
  
iterator &operator inc() {  
    ++y;  
    if (y == N) {  
        y = 0;  
        ++x;  
    }  
    advance_to_next();  
    return *this;  
}
```

- operator deref — разыменование
- operator arrow — доступ через указатель
- operator eq и operator ne — сравнение
- operator inc — инкремент (префиксный)

main()

main.cpp

```
int main() {
    bool data[4][4] = {
        {0, 1, 0, 0},
        {0, 0, 0, 1},
        {0, 0, 0, 0},
        {0, 0, 0, 1},
    };

    BinaryRelation<4> relation(data);

    std::cout << "Pairs in relation:\n";

    for (const auto &pair : relation) {
        std::cout << "(" << pair.first << ", " << pair.second << ")\n";
    }
}
```

Матрица смежности:

- (0,1) — связь 0 -> 1
- (1,3) — связь 1 -> 3
- (3,3) — связь 3 -> 3 (петля)

Вывод:

```
Pairs in relation:
(0, 1)
(1, 3)
(3, 3)
```

]

Вывод

В ходе лабораторной работы были изучены:

1. Шаблонный класс BinaryRelation — представление бинарного отношения
2. Пользовательский итератор — вложенный класс для обхода
3. Стандартные методы итераторов — begin, end, ++, ==, !=
4. For-each — использование в цикле for (const auto &pair : relation)

Итератор позволяет использовать объект в стиле STL и применять стандартные алгоритмы.